Technische Hochschule Ingolstadt



Machine Perception and Cognition: Stereo Estimation (Recap)

Epipolar Geometry — The Idea

- Two cameras observe the same 3D scene from different positions
- Given a pixel $\mathbf{p}_L$ in the **left** image: where is the matching pixel in the **right** image?
- **Naïve approach**: search every pixel in the right image → expensive and highly ambiguous
- **Epipolar insight**: the 3D point lies on a ray from the left camera center; this ray projects onto a **line** in the right image — the **epipolar line**
- The correct match $\mathbf{p}_R$ must lie on this epipolar line

This reduces a **2D search** to a **1D search** along the epipolar line — dramatically cutting computation and false matches.

Epipolar Geometry

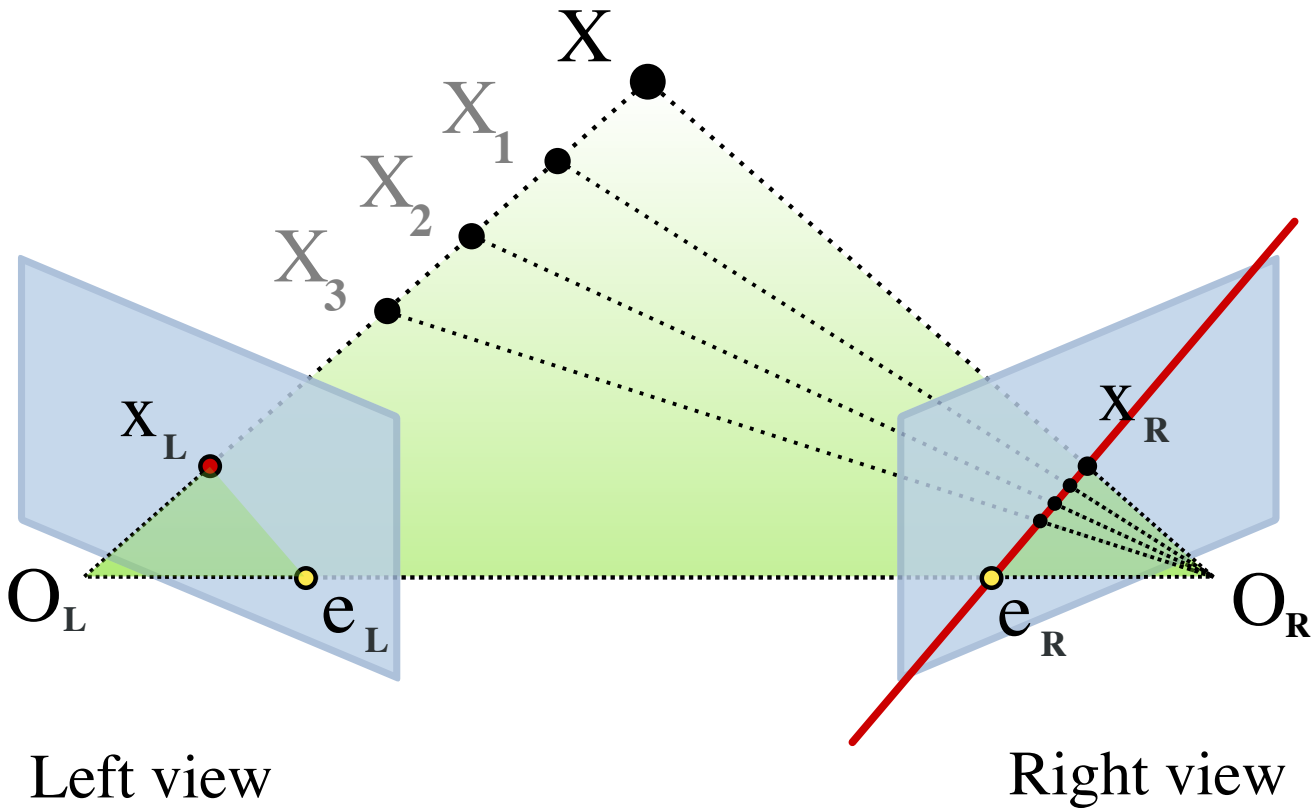


Image source: Arne Nordmann (norro), CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Epipolar_geometry.svg

The Epipoles

The **epipole** is the image of the other camera center, i.e. where the baseline (the line connecting both camera centers) intersects an image plane:

- $\mathbf{e}_L$ = projection of the **right** camera center O_R onto the **left** image
- $\mathbf{e}_R$ = projection of the **left** camera center O_L onto the **right** image

Every epipolar line in a given image passes through the epipole because every epipolar plane contains the baseline.

The Epipoles

Geometric interpretation:

- Fix a pixel in the left image. Its back-projection ray together with the baseline $O_L O_R$ defines an **epipolar plane**
- This plane intersects the right image plane in an **epipolar line**
- Because that plane contains O_L, the resulting line in the right image must pass through $\mathbf{e}_R$; symmetrically, epipolar lines in the left image pass through $\mathbf{e}_L$

This is the precise reason all epipolar lines share the epipole.

Epipolar Geometry

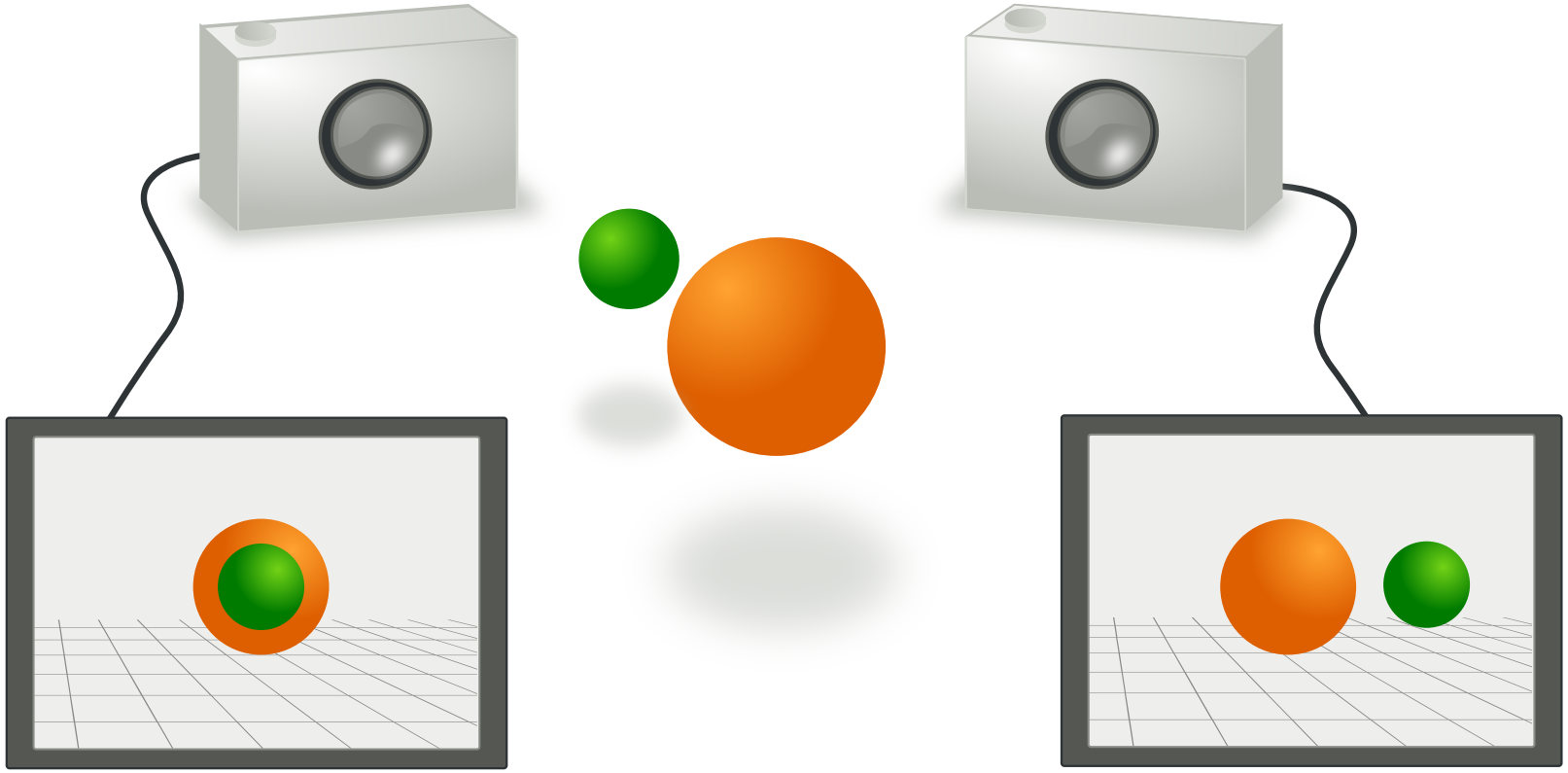


Image source: Arne Nordmann (norro), CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Aufnahme_mit_zwei_Kameras.svg

Why Epipolar Geometry Matters for Stereo Vision

- **Efficiency**: 1D line search instead of a 2D image search
- **Fewer false matches**: the epipolar constraint eliminates most incorrect candidates
- **Enables depth estimation**: for a **rectified, calibrated** stereo pair, once correspondences are found, depth follows from the horizontal disparity $d = u_L - u_R$, the baseline b, and the horizontal focal length f_x:

$$Z = \frac{f_x \cdot b}{d}$$

- Without the epipolar constraint, reliable dense stereo matching at scale would not be practical

Stereo Rectification — Why We Do It

In a general stereo setup, epipolar lines are **tilted** and do not align between the two images — searching along them is geometrically complex.

Rectification transforms both images so that:

- All epipolar lines become **horizontal rows**
- A point at row v in the left image can only match at the **same row** v in the right image

After rectification, stereo matching reduces to a simple **1D horizontal scan** per row — making disparity computation efficient and straightforward.

Depth from Disparity

For a rectified stereo pair, rearranging the stereo projection equations gives the **depth** Z:

$$Z = \frac{f_x \cdot b}{d}$$

This is the core formula of stereo estimation:

- f_x: horizontal focal length in pixels — from calibration
- b: baseline — from the stereo rig geometry
- $d = u_L - u_R$: horizontal disparity in pixels after rectification

The depth estimation problem reduces to accurately estimating disparity.

Focal Length in Pixels

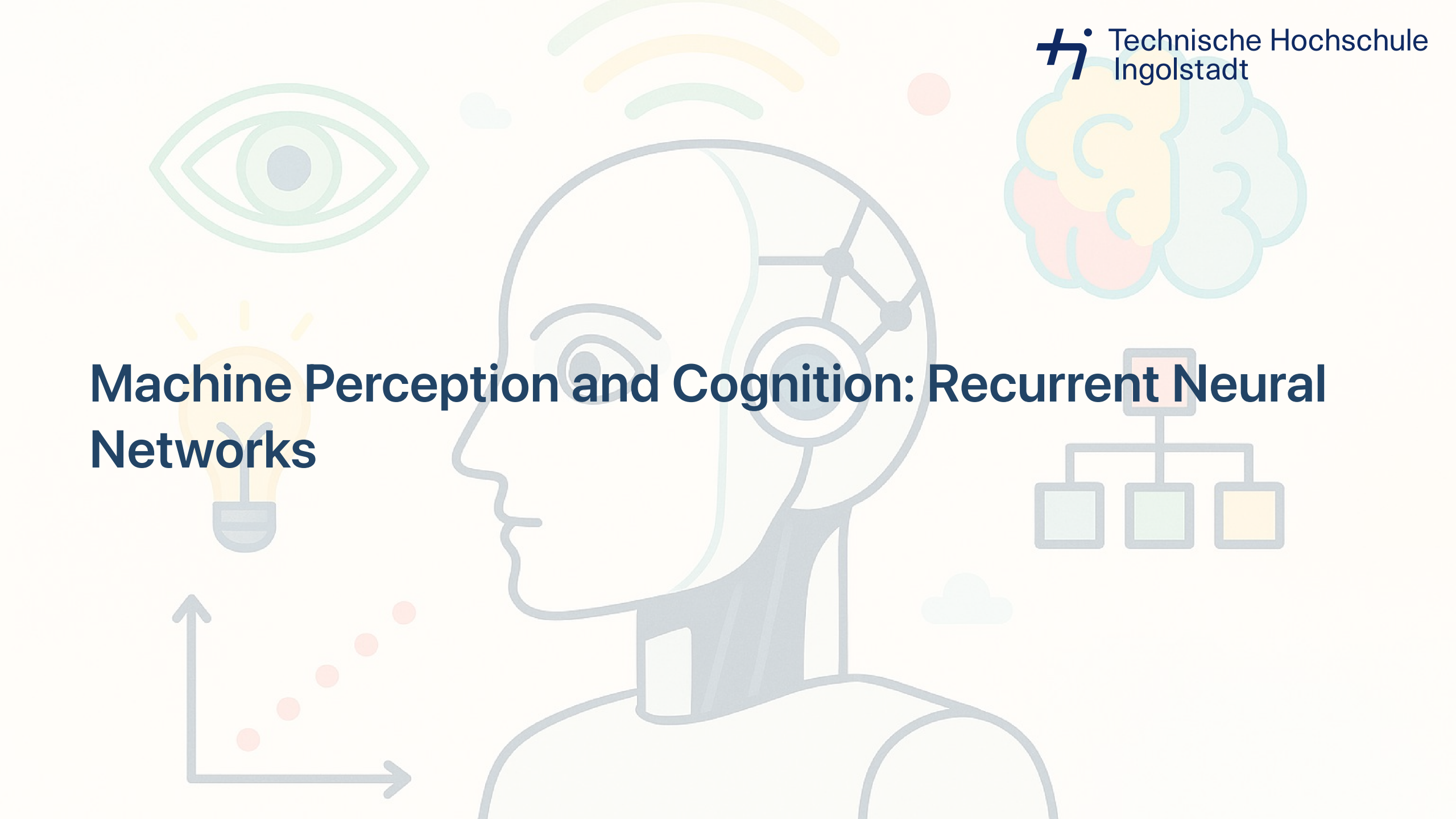
The focal length of a pinhole camera can be expressed in pixels instead of e.g. millimeters. For the horizontal image axis:

$$f_x = \frac{f_{mm}}{s_x}$$

where s_x is the horizontal pixel pitch in millimeters per pixel.

Thus $Z = \frac{f_x \cdot b}{d}$ works with f_x in pixels, d in pixels, and b in millimeters to retrieve depth Z in millimeters.

Technische Hochschule Ingolstadt



Machine Perception and Cognition: Recurrent Neural Networks

Sequential Data and Temporal Modeling

Not all data is independent — many real-world signals are **ordered sequences** where context matters:

- **Speech recognition**: the current phoneme depends on previous sounds
- **Video analysis**: the current frame depends on prior frames
- **Text**: the meaning of a word depends on preceding words
- **Sensor streams**: current robot state depends on past sensor readings

A standard feedforward network (MLP, CNN) receives a **fixed-size input** and maps it to an output — it has **no notion of time or sequence**.

We need a model that can **maintain memory across time steps**.

Limitations of Feedforward Networks for Sequences

A Multi-Layer Perceptron (MLP) processes each input independently:

$$\hat{\mathbf{y}} = \sigma_{\text{out}}(\mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$$

where σ_{out} is the output activation (e.g., softmax for classification, identity for regression).

Limitations of Feedforward Networks for Sequences

Problems for sequential data:

- No state is carried between time steps t and $t+1$
- To use temporal context, the entire sequence would need to be fed as one large input — fixing the maximum sequence length
- No **parameter sharing** across time: different weights would be needed for every time step
- Does not generalize to sequences longer than seen in training

What we need: a network that processes one time step at a time, but **maintains a hidden state** that summarizes the history.

The Idea of Recurrence

The key idea is to introduce a **recurrent connection**: the hidden state $\mathbf{h}_t$ at time t depends on the current input $\mathbf{x}_t$ **and** the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h)$$

- $\mathbf{h}_t \in \mathbb{R}^{n_h}$: hidden state vector at time t
- $\mathbf{W}_h \in \mathbb{R}^{n_h \times n_h}$: recurrent weight matrix
- $\mathbf{W}_x \in \mathbb{R}^{n_h \times n_x}$: input weight matrix
- σ: non-linear activation function (typically $\tanh$)

The **same weights $\mathbf{W}_h, \mathbf{W}_x$** are reused at every time step — this is **parameter sharing across time**.

Applications of Recurrent Neural Networks

RNNs can handle various **sequence input/output** relationships:

Task	Input	Output
Speech recognition	Audio frames (sequence)	Phonemes / words (sequence)
Sentiment analysis	Word sequence	Single label
Language modeling	Word sequence	Next word
Image captioning	Image	Word sequence
Machine translation	Source sentence	Target sentence
Video classification	Frame sequence	Single label

The flexibility to handle variable-length inputs and outputs makes RNNs suitable for many perception and cognition tasks.

Recurrent Neural Networks

Left as a cyclic graph, right unfolded over time:

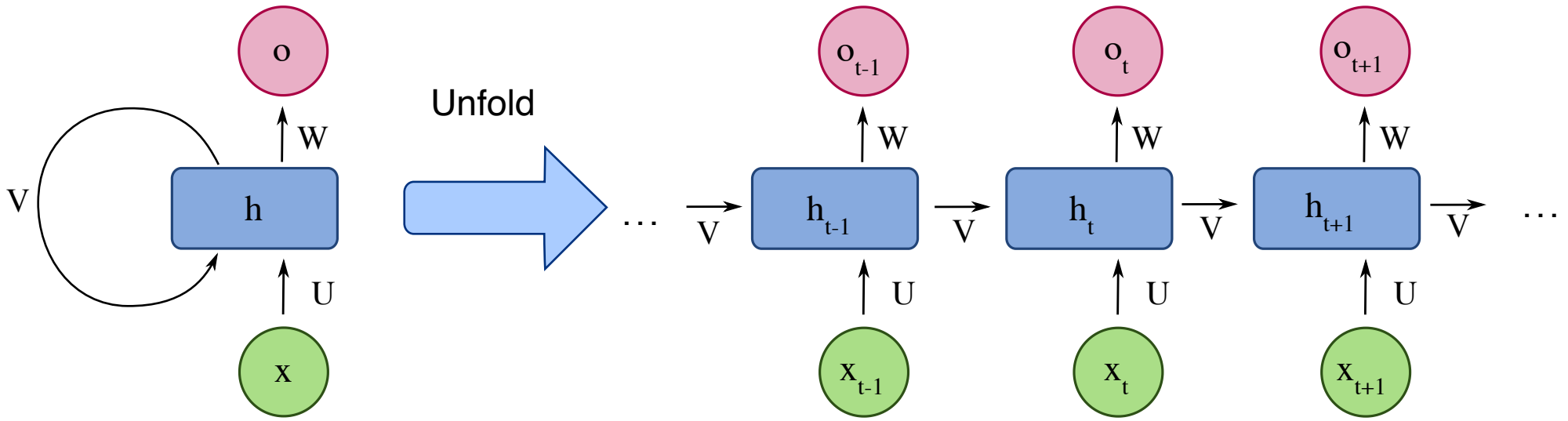


Image source: fdeloche, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Recurrent_neural_network_unfold.svg

RNN: Mathematical Formulation

An RNN processes a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$ step by step.

Hidden state update at each time step t:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h)$$

Output at each time step (if needed):

$$\hat{\mathbf{y}}_t = \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

Initial state: $\mathbf{h}_0 = \mathbf{0}$ (or learned)

The weight matrices $\mathbf{W}_h$, $\mathbf{W}_x$, $\mathbf{W}_y$ and biases $\mathbf{b}_h$, $\mathbf{b}_y$ are **shared across all time steps** — the model has the same number of parameters regardless of sequence length T.

RNN: The Hidden State as Memory

The hidden state $\mathbf{h}_t$ acts as a **compressed summary of the past**:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h)$$

At each step, $\mathbf{h}_t$ is influenced by:

- **The current input $\mathbf{x}_t$**: new information arriving at this step
- **The previous state $\mathbf{h}_{t-1}$**: accumulated information from all prior steps

Intuition:

- $\mathbf{W}_x$ controls how much of the new input is incorporated
- $\mathbf{W}_h$ controls how much of the past is remembered
- The $\tanh$ non-linearity squashes values to $(-1, 1)$, keeping the state bounded

RNN: Output and Loss

Depending on the task, outputs are extracted differently:

Many-to-many (sequence-to-sequence):

$$\hat{\mathbf{y}}_t = \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y), \quad t = 1, \ldots, T$$

Many-to-one (sequence classification) — only the final state is used:

$$\hat{\mathbf{y}} = \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_T + \mathbf{b}_y)$$

RNN: Output and Loss

The loss depends on the task:

Many-to-many:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(\hat{\mathbf{y}}_t, \mathbf{y}_t)$$

Many-to-one:

$$\mathcal{L} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$$

where $\mathcal{L}$ or $\mathcal{L}_t$ can be cross-entropy (classification) or mean squared error (regression).

Recurrent Neural Networks

Two RNN layers stacked:

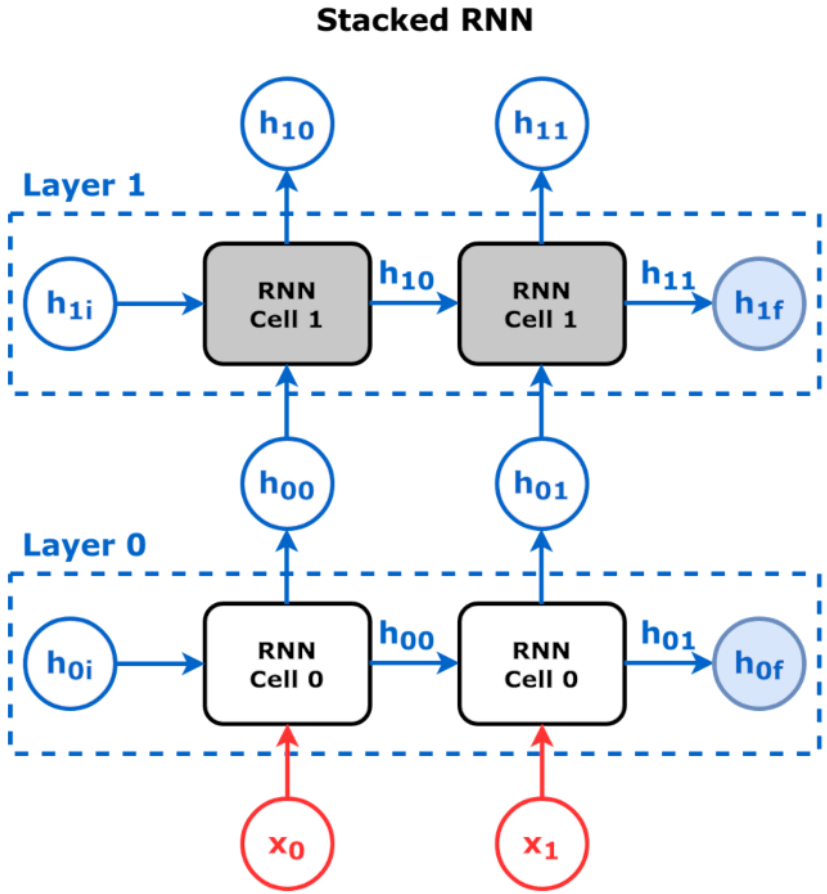


Image source: Daniel Voigt Godoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Stacked_RNN.png

Stacked RNNs

Multiple RNN layers can be stacked to increase **model capacity**:

- Layer 1 processes the input sequence $\mathbf{x}_1, \ldots, \mathbf{x}_T$ and outputs $\mathbf{h}_1^{(1)}, \ldots, \mathbf{h}_T^{(1)}$
- Layer 2 treats the outputs of layer 1 as its input sequence
- Each layer learns increasingly **abstract temporal features**

State update for layer ℓ:

$$\mathbf{h}_t^{(\ell)} = \tanh\left(\mathbf{W}_h^{(\ell)}\mathbf{h}_{t-1}^{(\ell)} + \mathbf{W}_x^{(\ell)}\mathbf{h}_t^{(\ell-1)} + \mathbf{b}_h^{(\ell)}\right)$$

Bidirectional RNN: Motivation

A standard (unidirectional) RNN processes the sequence **left to right**: at step t, it can only use information from $\mathbf{x}_1, \ldots, \mathbf{x}_t$.

However, for many tasks the **future context** is also informative:

- In speech recognition: the pronunciation of a phoneme depends on surrounding sounds
- In named entity recognition: a word's label depends on both preceding and following words
- In text classification: a word near the end can inform the meaning of earlier words

Bidirectional RNN: Motivation

A **Bidirectional RNN (Bi-RNN)** runs two separate RNNs in parallel (not stacked):

- A **forward RNN** from $t = 1$ to T
- A **backward RNN** from $t = T$ to 1

Their outputs are concatenated at each step, giving access to the **full sequence context**.

Recurrent Neural Networks

Bidirectional RNN:

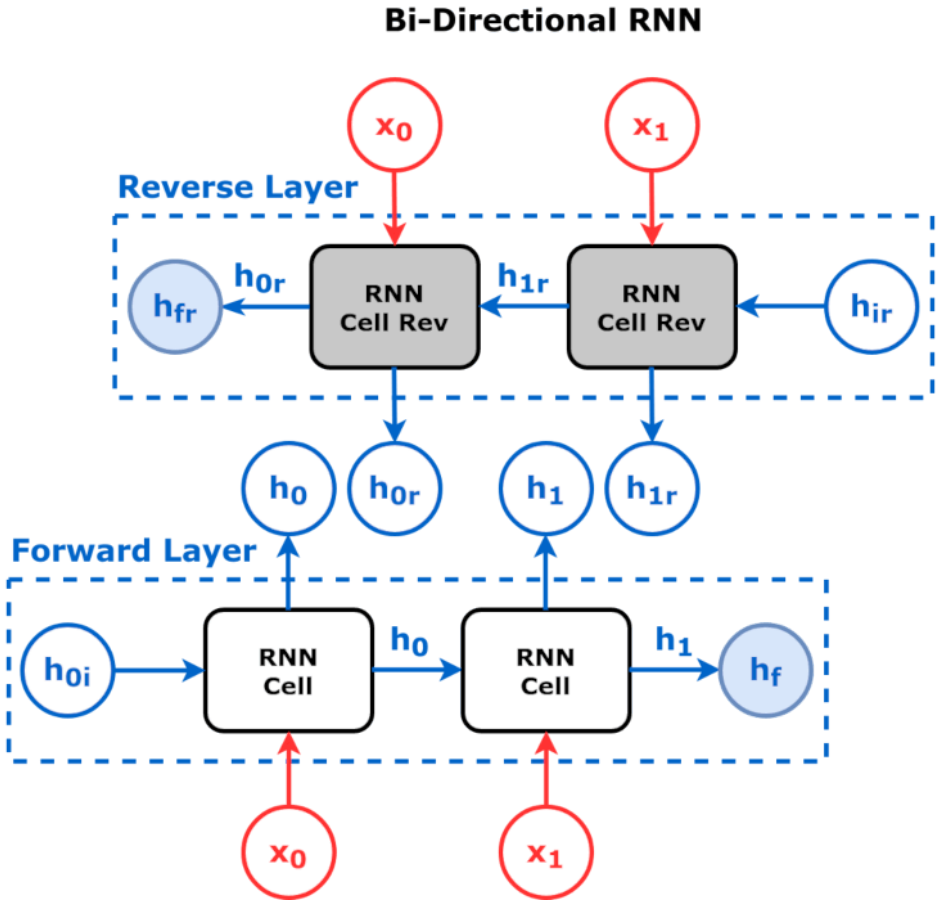


Image source: Daniel Voigt Godoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Bidirectional_RNN.png

Bidirectional RNN: Equations

Forward pass (left to right):

$$\overrightarrow{\mathbf{h}}_t = \tanh\left(\mathbf{W}_{\overrightarrow{h}}\overrightarrow{\mathbf{h}}_{t-1} + \mathbf{W}_{\overrightarrow{x}}\mathbf{x}_t + \mathbf{b}_{\overrightarrow{h}}\right)$$

Backward pass (right to left):

$$\overleftarrow{\mathbf{h}}_t = \tanh\left(\mathbf{W}_{\overleftarrow{h}}\overleftarrow{\mathbf{h}}_{t+1} + \mathbf{W}_{\overleftarrow{x}}\mathbf{x}_t + \mathbf{b}_{\overleftarrow{h}}\right)$$

Combined output at time t:

$$\hat{\mathbf{y}}_t = \sigma_{\text{out}}\left(\mathbf{W}_y \begin{bmatrix} \overrightarrow{\mathbf{h}}_t \\ \overleftarrow{\mathbf{h}}_t \end{bmatrix} + \mathbf{b}_y\right)$$



Bidirectional RNN: Limitation

Standard bidirectional RNNs require the **entire sequence** to be available before processing — this makes them unsuitable for strictly causal real-time or streaming inference.

Image source: https://www.pexels.com/photo/yellow-and-black-striped-rod-sign-18318662/

Training RNNs: Backpropagation Through Time

To train an RNN, we need gradients of the loss $\mathcal{L}$ with respect to all weights: $\mathbf{W}_h$, $\mathbf{W}_x$, $\mathbf{W}_y$.

Because the weights are **shared across time**, the gradient at any step depends on all earlier steps (working backwards).

Idea: Unroll the RNN for T steps and apply standard backpropagation on the unrolled computation graph. This algorithm is called **Backpropagation Through Time (BPTT)**.

The unrolled graph is a deep feedforward network where:

- Each layer corresponds to one time step
- Weights are **tied** (shared) across all layers

BPTT computes exact gradients — but becomes **expensive** for long sequences.

BPTT: Unrolling Through Time

Consider the RNN unrolled for T time steps. The loss at each step t is $\mathcal{L}_t$, and the total loss is $\mathcal{L} = \sum_t \mathcal{L}_t$.

The gradient of $\mathcal{L}$ with respect to $\mathbf{W}_h$ accumulates contributions from every step:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_h}$$

Each term $\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_h}$ requires tracing back through all hidden states from step t down to step 1.

Key insight: The gradient at step t depends on the **entire history** of hidden states $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_t$ — this is what makes training RNNs on long sequences challenging.

BPTT: The Chain Rule Across Time

The gradient of the hidden state at step t with respect to the hidden state at an earlier step $k \leq t$ involves a **product of Jacobians**:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{h}_{t-2}} \cdots \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k}$$

This product of $(t - k)$ matrices is the **core of the vanishing/exploding gradient problem** — its norm either shrinks or grows exponentially with the number of steps.

Truncated Backpropagation Through Time

Full BPTT over very long sequences ($T \gg 100$) is:

- **Memory-intensive**: all intermediate activations must be stored
- **Computationally expensive**: gradients must be propagated T steps back

Truncated BPTT (TBPTT) is a practical approximation:

1. Process the sequence in chunks of fixed length k
2. Forward pass through k steps, keeping the hidden state
3. Backpropagate gradients only through the last k steps
4. Move the window forward and repeat

Trade-off: TBPTT makes learning dependencies much longer than k steps difficult, but significantly reduces memory and computation.

The Vanishing Gradient Problem

From the BPTT chain rule, the gradient of the loss at step t with respect to $\mathbf{W}_h$ requires:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_h} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_h}$$

For a simple $\tanh$ RNN, each Jacobian contains both the activation derivative and the recurrent weights, e.g. $\frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \operatorname{diag}(1 - \mathbf{h}_i \odot \mathbf{h}_i)\mathbf{W}_h$. Repeated multiplication can therefore make the gradient norm decay rapidly.

The Vanishing Gradient Problem

Consequence: If the relevant Jacobian norms are typically < 1, the product $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ shrinks exponentially with $(t - k)$.

For early steps $k \ll t$, the gradient **vanishes** — the weight update receives no signal from distant past inputs, making long-range dependencies impossible to learn.

Vanishing Gradient: The Chain Rule in Detail

To make this concrete, consider the scalar approximation — a single hidden unit with weight w_h:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} \left(\tanh'(a_i) \cdot w_h \right)$$

where $a_i = w_h h_{i-1} + w_x x_i + b$.

Vanishing Gradient: The Chain Rule in Detail

Let $\alpha = \tanh'(a_i) \cdot w_h$ be an approximately constant per-step factor:

$$\frac{\partial h_t}{\partial h_k} \approx \alpha^{t-k}$$

- If $|\alpha| < 1$: the partial derivative product **vanishes** — $\alpha^{t-k} \to 0$ as $(t-k)$ grows:

$$0.9^{20} \approx 0.12 \qquad 0.9^{50} \approx 0.005 \qquad 0.9^{100} \approx 0.00003$$

The Exploding Gradient Problem

The same chain rule product leads to the **opposite** failure when the relevant Jacobian norms are persistently > 1:

$$\frac{\partial h_t}{\partial h_k} \approx \alpha^{t-k}, \quad |\alpha| > 1$$

The partial derivative product then **explodes** exponentially:

$$1.1^{20} \approx 6.7 \qquad 1.1^{50} \approx 117 \qquad 1.1^{100} \approx 13780$$

Long-Range Dependencies: The Core Challenge

The fundamental challenge for simple RNNs:

Example (language): "The cat that chased all the mice in the barn … **was** old."

To correctly predict "was" (singular), the model must remember "cat" from many steps ago — despite many intervening words.

Why simple RNNs fail:

- The hidden state $\mathbf{h}_t$ is overwritten at every step through a saturating non-linearity
- Gradient signal vanishes over many steps during BPTT
- The recurrent weight matrix $\mathbf{W}_h$ cannot simultaneously preserve long-term information and stay well-conditioned

Long-Range Dependencies: The Core Challenge

The solution: An architecture with **explicit, gated memory** that can selectively read, write, and erase information across time.

This is the **Long Short-Term Memory (LSTM)** cell — introduced by Hochreiter & Schmidhuber (1997).

LSTM: Motivation and Overview

The Long Short-Term Memory (LSTM) was designed to solve the **vanishing gradient problem** and enable learning of **long-range dependencies**.



Image source: https://www.pexels.com/photo/turned-on-light-bulb-712490/

LSTM: Motivation and Overview

Key innovations over simple RNNs:

1. A **cell state** $\mathbf{c}_t$: a separate, slowly-varying memory that flows through time with minimal modification — acting like a "conveyor belt"
2. **Gates**: learned mechanisms that control the flow of information:
    - **Forget gate**: what to erase from memory
    - **Input gate**: what new information to write
    - **Output gate**: what to expose as the hidden state

Each gate is a learned sigmoid function — outputting values in $(0, 1)$ — acting as a **soft switch**.

An LSTM cell maintains **two** state vectors: the cell state $\mathbf{c}_t$ and the hidden state $\mathbf{h}_t$.

Long Short-Term Memory Cells

A LSTM cell with "peephole" (cell state can be seen by gates):

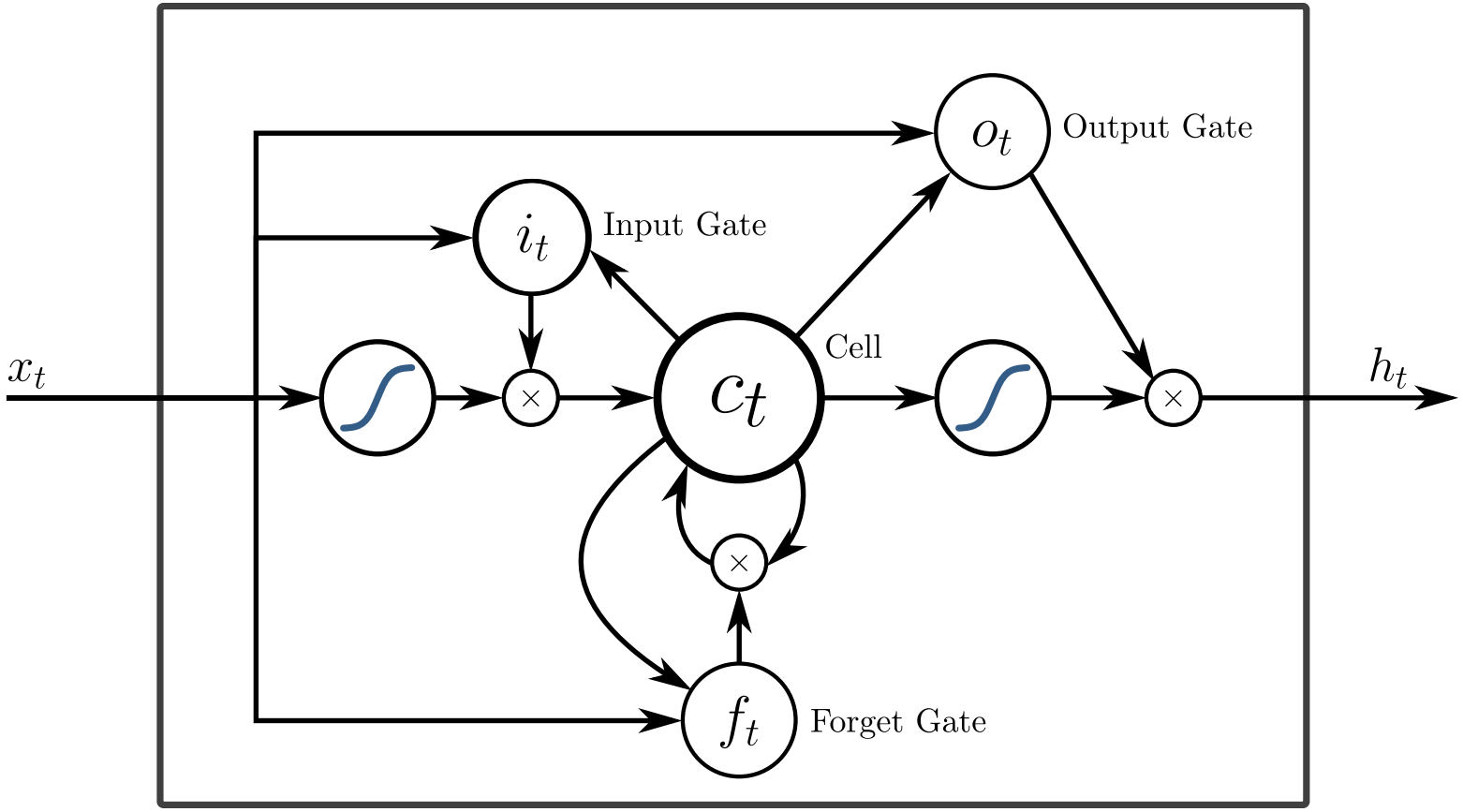


Image source: https://commons.wikimedia.org/wiki/User:BiObserve (Raster version previously uploaded to Wikimedia); Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton (original); Eddie Antonio Santos (SVG version with TeX math), CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Peephole_Long_Short-Term_Memory.svg

LSTM: The Cell State

The **cell state** $\mathbf{c}_t \in \mathbb{R}^{n_h}$ is the LSTM's long-term memory.

Unlike the hidden state, the cell state is modified only through **additive updates** controlled by gates:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$$

- $\mathbf{f}_t$: forget gate — how much of the **old memory** to keep
- $\mathbf{i}_t$: input gate — how much of the **candidate memory** to add
- $\mathbf{g}_t$: candidate cell state — proposed new memory content
- $\odot$: element-wise (Hadamard) product

Why this helps with vanishing gradients: The cell state is updated **additively** rather than through repeated matrix multiplications. Gradient information can flow directly back through time along the cell state path without being multiplied by $\mathbf{W}_h$ at every step.

LSTM: Forget Gate

The **forget gate** $\mathbf{f}_t$ decides what information to discard from the cell state:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{h}_{t-1} + \mathbf{U}_f \mathbf{x}_t + \mathbf{b}_f)$$

- σ: sigmoid function — outputs values in $(0, 1)$
- $\mathbf{f}_t \approx 0$: **forget** the corresponding cell state dimension
- $\mathbf{f}_t \approx 1$: **keep** the corresponding cell state dimension

The forget gate is applied element-wise to the previous cell state:

$$\mathbf{c}_{t-1}^{\text{gated}} = \mathbf{f}_t \odot \mathbf{c}_{t-1}$$

Example: In a language model, when reading a new subject noun, the forget gate can reset the cell state entry that stored the previous subject's gender/number agreement.

LSTM: Input Gate and Candidate Memory

The **input gate** $\mathbf{i}_t$ decides what new information to store:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{h}_{t-1} + \mathbf{U}_i \mathbf{x}_t + \mathbf{b}_i)$$

The **candidate cell state** $\mathbf{g}_t$ is the proposed new memory content:

$$\mathbf{g}_t = \tanh(\mathbf{W}_c \mathbf{h}_{t-1} + \mathbf{U}_c \mathbf{x}_t + \mathbf{b}_c)$$

Together they update the cell state:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$$

- $\mathbf{i}_t \approx 0$: **ignore** new input for this dimension
- $\mathbf{i}_t \approx 1$: **write** new candidate value into memory

The input gate and forget gate work together: the cell can **selectively erase old information and write new information** at each step.

LSTM: Output Gate

The **output gate** $\mathbf{o}_t$ controls what part of the cell state is exposed as the hidden state:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{h}_{t-1} + \mathbf{U}_o \mathbf{x}_t + \mathbf{b}_o)$$

The hidden state $\mathbf{h}_t$ is then:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- $\tanh(\mathbf{c}_t)$ squashes the cell state to $(-1, 1)$
- $\mathbf{o}_t$ gates which dimensions are exposed
- $\mathbf{o}_t \approx 0$: **hide** this dimension from the output
- $\mathbf{o}_t \approx 1$: **expose** this dimension as the hidden state

The hidden state $\mathbf{h}_t$ is passed to the next layer and used for predictions. The cell state $\mathbf{c}_t$ carries long-term information forward.

LSTM: Complete Equations

Given input $\mathbf{x}_t$, previous hidden state $\mathbf{h}_{t-1}$, and previous cell state $\mathbf{c}_{t-1}$:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{h}_{t-1} + \mathbf{U}_f \mathbf{x}_t + \mathbf{b}_f) \quad \text{(forget gate)}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{h}_{t-1} + \mathbf{U}_i \mathbf{x}_t + \mathbf{b}_i) \quad \text{(input gate)}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{h}_{t-1} + \mathbf{U}_o \mathbf{x}_t + \mathbf{b}_o) \quad \text{(output gate)}$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_c \mathbf{h}_{t-1} + \mathbf{U}_c \mathbf{x}_t + \mathbf{b}_c) \quad \text{(candidate memory)}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad \text{(cell state update)}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden state / output)}$$

LSTM: Why Gradients Flow Better

The key is the **additive cell state update**:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$$

Simple RNN: the hidden state is overwritten at every step via a matrix multiplication and a saturating non-linearity ($\tanh$). Gradients must pass through this operation at *every* time step — causing them to shrink (or grow) exponentially over many steps.

LSTM: the cell state $\mathbf{c}_t$ is updated *additively* — old memory is scaled and new information is added on top. Gradient information can flow back through time along this additive path without being repeatedly squeezed by a saturating non-linearity.

LSTM: Why Gradients Flow Better

Key insight:

- When the forget gate stays close to 1 (keep information), past cell states are passed through nearly unchanged — and so are the gradients
- The LSTM can learn **when to remember** and **when to forget**, thereby controlling its own gradient flow

This is fundamentally different from the simple RNN, where gradients must be repeatedly multiplied through the recurrent weight matrix at every step.

Long Short-Term Memory Cells

A LSTM cell with "peephole" (cell state can be seen by gates):

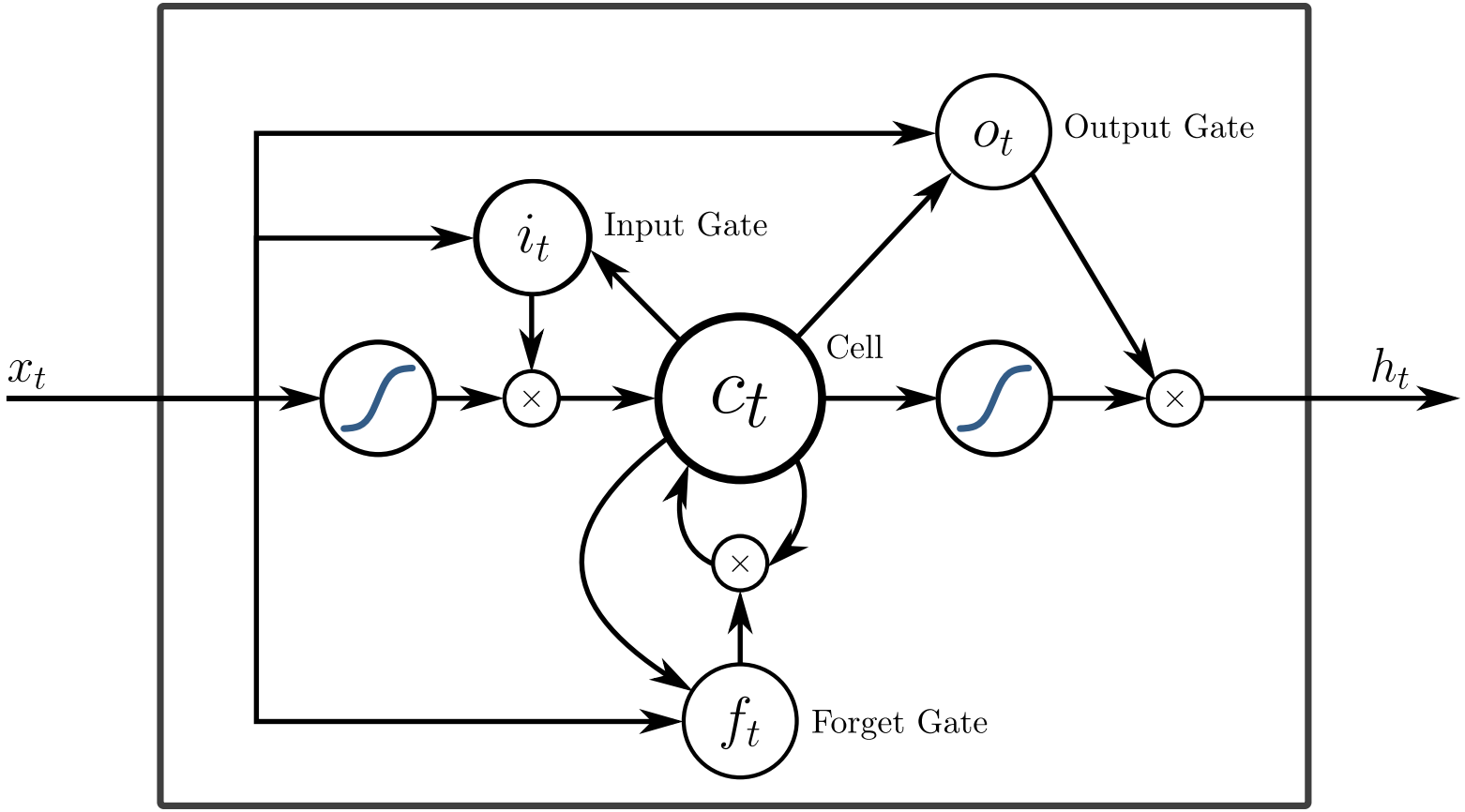


Image source: https://commons.wikimedia.org/wiki/User:BiObserve (Raster version previously uploaded to Wikimedia); Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton (original); Eddie Antonio Santos (SVG version with TeX math), CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Peephole_Long_Short-Term_Memory.svg

LSTM with Peephole Connections

Peephole connections (Gers & Schmidhuber, 2000) allow the gates to directly **observe the cell state** (typically $\mathbf{c}_{t-1}$ for forget/input and $\mathbf{c}_t$ for output):

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{h}_{t-1} + \mathbf{U}_f \mathbf{x}_t + \mathbf{P}_f \odot \mathbf{c}_{t-1} + \mathbf{b}_f)$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{h}_{t-1} + \mathbf{U}_i \mathbf{x}_t + \mathbf{P}_i \odot \mathbf{c}_{t-1} + \mathbf{b}_i)$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{h}_{t-1} + \mathbf{U}_o \mathbf{x}_t + \mathbf{P}_o \odot \mathbf{c}_t + \mathbf{b}_o)$$

where $\mathbf{P}_f, \mathbf{P}_i, \mathbf{P}_o$ are **peephole weight vectors** (equivalently diagonal matrices, one scalar per hidden dimension).

Motivation: Without peepholes, the gates can only indirectly observe the cell state through $\mathbf{h}_{t-1} = \mathbf{o}_{t-1} \odot \tanh(\mathbf{c}_{t-1})$. Peepholes give the gates more direct access to what is actually stored in memory.

Long Short-Term Memory Cells

A LSTM cell unrolled with time indexes:

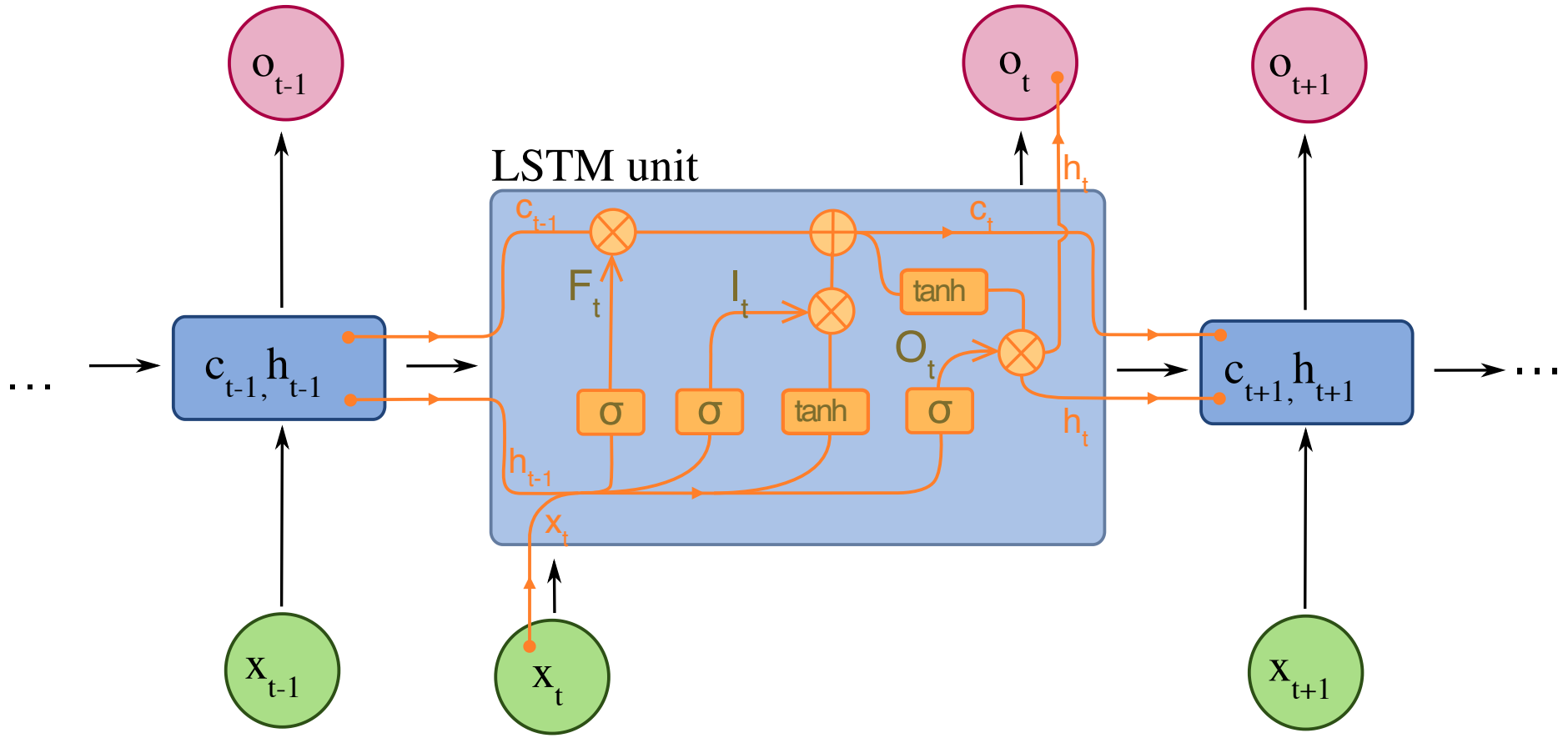


Image source: fdeloche, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Long_Short-Term_Memory.svg

Gated Recurrent Unit (GRU)

The **Gated Recurrent Unit** (GRU, Cho et al., 2014) is a simplified alternative to LSTM:

Update gate (interpolates between old state and candidate):

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{h}_{t-1} + \mathbf{U}_z \mathbf{x}_t + \mathbf{b}_z)$$

Reset gate (controls how much past state is used for the candidate):

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{h}_{t-1} + \mathbf{U}_r \mathbf{x}_t + \mathbf{b}_r)$$

Candidate hidden state:

$$\mathbf{n}_t = \tanh(\mathbf{W}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{U}_h \mathbf{x}_t + \mathbf{b}_h)$$

Hidden state update:

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \mathbf{n}_t$$

GRU has **no separate cell state** — only one state vector $\mathbf{h}_t$.

Comparing RNN Variants

Feature	Simple RNN	LSTM	GRU
State vectors	$\mathbf{h}_t$ only	$\mathbf{h}_t$ and $\mathbf{c}_t$	$\mathbf{h}_t$ only
Number of gates	—	3 (forget, input, output)	2 (update, reset)
Long-range dependencies	Poor	Good	Good
Vanishing gradient	Severe	Mitigated	Mitigated

Recap!

Recurrent Neural Networks (RNNs):

- Maintain a hidden state $\mathbf{h}_t$ that summarizes the history of the sequence
- Process sequences with **shared weights** across time steps
- Trained with **Backpropagation Through Time (BPTT)**

Bidirectional RNNs:

- Access both past and future context by running two RNNs in parallel

Recap!

Vanishing and Exploding Gradients:

- Simple RNNs struggle with long-range dependencies due to gradient decay
- Vanishing gradients are addressed by **gated architectures**

Long Short-Term Memory (LSTM):

- Introduces a **cell state** and three gates (forget, input, output)
- Enables learning of **long-range dependencies** via additive cell state updates
- **Peephole connections** allow gates to directly observe the cell state